Here is a detailed breakdown of each program, explaining the underlying logic, the data structures used, and the algorithmic flow based on the provided source code.

---

1. Breadth First Search (BFS) & Depth First Search (DFS)

Domain: Tree Traversal and Graph Theory.

This program demonstrates the four primary ways to visit every node in a binary tree.

- **Tree Structure:** Uses a Node class with data, left, and right pointers.
- **DFS (Depth First Search):** Implemented recursively, where the algorithm "dives" deep into the left subtree before visiting the right.
    - **Pre-order:** Processes the root first. Useful for creating a copy of the tree.
    - **In-order:** Processes the left child, then root, then right child. In a Binary Search Tree, this returns values in ascending order.
    - **Post-order:** Processes children before the root. Ideal for deleting a tree or calculating its size.
- **BFS (Breadth First Search):** Implemented using a Queue. It visits nodes level-by-level (top to bottom, left to right), ensuring that all nodes at distance d are visited before nodes at distance $d+1$.

---

2. A* Algorithm (8-Puzzle Problem)

Domain: Artificial Intelligence / Heuristic Search.

The A* algorithm solves the 8-puzzle by navigating a state-space tree to find the configuration with the lowest "estimated cost".

- **Heuristic Function ($f(n) = g(n) + h(n)$):** * $g(n)$ is the level1 variable, representing the number of moves made from the start.
    - $h(n)$ is the cost1, calculated by calculateCost(), which counts how many tiles are not in their goal positions.
- **Priority Queue:** The code uses a custom Comparator to always select the node where $f(n)$ is minimized.
- **State Exploration:** For every move, it generates up to four child nodes (Up, Down, Left, Right) by swapping the blank tile (0) with an adjacent tile.

---

3. Selection Sort

Domain: Sorting Algorithms.

Selection sort is a comparison-based algorithm that focuses on finding the extreme value (minimum) in each iteration.

- **Pass-by-Pass Logic:** The outer loop runs n times. For each index i, the inner loop searches the remainder of the array to find the index of the smallest element (minIdx).
- **Swapping:** Once the minimum is found, it is swapped with the element at index i.
- **In-place Sorting:** The algorithm is "in-place," meaning it requires only a constant amount $O(1)$ of extra memory space beyond the original array.

---

4. N-Queens Problem

Domain: Backtracking.

This program solves the challenge of placing N queens on a board so that no two queens attack each other.

- **Backtracking Strategy:** The helper function tries to place a queen in the first row of a column and moves to the next column.
- **Conflict Detection:** The safe() function checks three directions: the horizontal row to the left, the upper-left diagonal, and the lower-left diagonal. (Checking the right is unnecessary because queens are placed left-to-right).
- **Recursion:** If a queen cannot be placed in any row of the current column, the function returns false, prompting the previous recursive call to move its queen and try again.

---

5. Simple Chatbot

Domain: Human-Computer Interaction.

This is a **Rule-Based Expert System** in its simplest form, using keyword matching to drive conversation.

- **Input Processing:** The bot takes userInput and converts it to lower-case (via equalsIgnoreCase or contains) to maintain case-insensitivity.

- **Response Mapping:** It uses a series of if-else blocks to act as a knowledge base. For example, if the keyword "fees" is detected, it triggers a specific string response regarding admissions.
- **Termination:** The loop continues until the specific sentinel value "bye" is entered.

---

6. Expert System (Stock Market)

Domain: Knowledge-Based Systems.

This program simulates an expert financial advisor using **Forward Chaining** logic.

- **User Interface:** The system gathers "facts" from the user through the askQuestion method.
- **Inference Engine:** The Evaluate function acts as the inference engine. It applies a strict Boolean rule: $Trend(Upwards) \land Fundamentals(Strong) \land Indicators(Positive) \implies Recommendation(Buy)$.
- **Domain Specificity:** Unlike the chatbot, this system is focused on a singular task: providing a binary decision based on specific domain knowledge.

---

7. Diffie-Hellman Key Exchange

Domain: Network Security / Cryptography.

This script implements a protocol to establish a shared secret over an insecure channel.

- **Public Parameters:** Both parties agree on a large prime n and a primitive root g.
- **Private Keys:** Alice and Bob choose secret integers x and y.
- **Mathematical Exchange:**
    - Alice sends $A = g^x \pmod n$.
    - Bob sends $B = g^y \pmod n$.
- **Key Derivation:** Alice computes $k_1 = B^x \pmod n$ and Bob computes $k_2 = A^y \pmod n$. Due to modular exponentiation properties, $k_1 = k_2 = g^{xy} \pmod n$.

---

8. RSA Algorithm

Domain: Asymmetric Encryption.

The RSA program demonstrates the creation of a Public/Private key pair based on the difficulty of integer factorization.

- **Key Generation:**
    1. Selects $p=7, q=17$, so $n=119$.
    2. Calculates $\phi(n) = (p-1)(q-1) = 96$.
    3. Finds e (public exponent) such that $gcd(e, 96) = 1$.
    4. Calculates d (private exponent) as the modular multiplicative inverse of $e \pmod{96}$.
- **Encryption/Decryption:**
    - $Ciphertext = Message^e \pmod n$.
    - $Plaintext = Ciphertext^d \pmod n$.

---

9. Transposition Cipher

Domain: Classical Cryptography.

This program implements a **Columnar Transposition Cipher**, which secures data by changing the position of characters.

- **Encryption:** The text is written into a grid based on the key (number of columns). The ciphertext is then read column-by-column.
- **Decryption:** This is more complex because it requires calculating the number of "rows" and "shaded boxes" (empty spaces in the grid) to correctly map the ciphertext back into the grid structure.
- **Logic:** It uses math.ceil(len(message) / key) to determine the grid dimensions.

---

10. XOR and AND Operation

Domain: Bitwise Security Operations.

This program manipulates the binary representation of a string to demonstrate basic data obfuscation.

- **AND Operation:** Applying & 127 is a bit-masking technique. Since 127 is 01111111 in binary, it ensures that only the first 7 bits of any character are kept, which effectively strips the 8th bit (often used in extended ASCII).
- **XOR Operation:** Applying ^ 127 flips the bits of the character. In cryptography, XOR is vital because it is reversible and does not leak the frequency of the original bits as easily as simple addition.
- **Conversion:** ord(char) gets the integer code, and chr() converts the modified integer back into a character.